Data Structures — Complete Answers with Code & I/O

SET 01 — Q1: Total and Average of N Students Using Structures

Description

A structure `Student` stores roll number, name, and marks. We loop through `n` students, read their data, compute total marks, and calculate average.

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student s[n];
    float total = 0;

    for (int i = 0; i < n; i++) {
        printf("\nEnter Roll No: ");
        scanf("%d", &s[i].roll);
        printf("Enter Name: ");
        scanf("%s", s[i].name);
        printf("Enter Marks: ");
        scanf("%f", &s[i].marks);
        total += s[i].marks;
    }

    printf("\n%-5s %-15s %-10s\n", "Roll", "Name", "Marks");
    printf("-------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-5d %-15s %-10.2f\n", s[i].roll, s[i].name,
s[i].marks);
    }

    printf("\nTotal Marks  : %.2f", total);
    printf("\nAverage Marks: %.2f\n", total / n);
    return 0;
}
```

Sample Input

```
Enter number of students: 3
Enter Roll No: 1   Enter Name: Alice   Enter Marks: 85
Enter Roll No: 2   Enter Name: Bob     Enter Marks: 90
Enter Roll No: 3   Enter Name: Carol   Enter Marks: 78
```

Sample Output

```
Roll  Name            Marks
```

```
-----------------------------
1     Alice           85.00
2     Bob             90.00
3     Carol           78.00

Total Marks  : 253.00
Average Marks: 84.33
```

SET 01 — Q2: Merge Sort Using Arrays

Description

Merge Sort is a divide-and-conquer algorithm. The array is split into halves recursively until single elements remain, then merged back in sorted order. Time: O(n log n).

```c
#include <stdio.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Sample Input

```
Enter number of elements: 6
Enter elements: 38 27 43 3 9 82
```

Sample Output

```
Sorted array: 3 9 27 38 43 82
```

SET 02 — Q1: Evaluate Postfix Expression

Description

In postfix (Reverse Polish Notation), operators follow their operands. A stack is used: scan left to right — push operands, pop two operands when operator found, compute, push result.

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int stack[100];
int top = -1;

void push(int val)  { stack[++top] = val; }
int  pop()          { return stack[top--]; }

int main() {
    char expr[100];
    printf("Enter postfix expression: ");
    scanf("%s", expr);

    for (int i = 0; i < strlen(expr); i++) {
        char ch = expr[i];
        if (isdigit(ch)) {
            push(ch - '0');
        } else {
            int b = pop(), a = pop();
            switch (ch) {
                case '+': push(a + b); break;
                case '-': push(a - b); break;
                case '*': push(a * b); break;
                case '/': push(a / b); break;
            }
        }
    }
    printf("Result: %d\n", pop());
    return 0;
}
```

Sample Input

```
Enter postfix expression: 231*+9-
```

*(Expression: 2 + 31 - 9 = -4)**

Sample Output

```
Result: -4
```

SET 02 — Q2: Quick Sort Using Arrays

Description

Quick Sort picks a pivot element and partitions the array such that elements less than the pivot come before it and greater come after. Recursively sort the two sub-arrays. Average Time: O(n log n).

```c
#include <stdio.h>

void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int partition(int arr[], int low, int high) {
```

```c
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Sample Input

```
Enter number of elements: 5
Enter elements: 64 34 25 12 22
```

Sample Output

```
Sorted array: 12 22 25 34 64
```

SET 03 — Q1: Student Details from Command Line

Description

Uses `argc` and `argv[]` to accept student name and marks directly from the command line at run-time. Marks are parsed and totaled from the arguments.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <name> <mark1> <mark2> ...\n", argv[0]);
        return 1;
    }

    printf("Student Name : %s\n", argv[1]);
    float total = 0;
    int count = argc - 2;
```

```c
        printf("Marks        : ");
        for (int i = 2; i < argc; i++) {
            float m = atof(argv[i]);
            printf("%.2f ", m);
            total += m;
        }

        printf("\nTotal Marks  : %.2f\n", total);
        printf("Average Marks: %.2f\n", total / count);
        return 0;
    }
```

Sample Input (command line)

```
./student Alice 85 90 78 92
```

Sample Output

```
Student Name : Alice
Marks        : 85.00 90.00 78.00 92.00
Total Marks  : 345.00
Average Marks: 86.25
```

SET 03 — Q2: Radix Sort Using Arrays

Description

Radix Sort is a non-comparison sort. It sorts numbers digit by digit from least significant to most significant using counting sort as a subroutine. Time: O(d*(n+b)) where d = digits, b = base (10).

```c
    #include <stdio.h>

    int getMax(int arr[], int n) {
        int mx = arr[0];
        for (int i = 1; i < n; i++) if (arr[i] > mx) mx = arr[i];
        return mx;
    }

    void countSort(int arr[], int n, int exp) {
        int output[n], count[10] = {0};
        for (int i = 0; i < n; i++) count[(arr[i] / exp) % 10]++;
        for (int i = 1; i < 10; i++) count[i] += count[i - 1];
        for (int i = n - 1; i >= 0; i--) {
            output[--count[(arr[i] / exp) % 10]] = arr[i];
        }
        for (int i = 0; i < n; i++) arr[i] = output[i];
    }

    void radixSort(int arr[], int n) {
        int m = getMax(arr, n);
        for (int exp = 1; m / exp > 0; exp *= 10)
            countSort(arr, n, exp);
    }

    int main() {
        int n;
        printf("Enter number of elements: ");
        scanf("%d", &n);
        int arr[n];
        printf("Enter elements: ");
        for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

        radixSort(arr, n);
```

```
            printf("Sorted array: ");
            for (int i = 0; i < n; i++) printf("%d ", arr[i]);
            printf("\n");
            return 0;
        }
```

Sample Input

```
Enter number of elements: 6
Enter elements: 170 45 75 90 802 24
```

Sample Output

```
Sorted array: 24 45 75 90 170 802
```

SET 04 — Q1: Stack Using Arrays (Push, Pop, Peek, Display)

Description

A stack is a LIFO (Last In First Out) data structure. We use an array with a `top` pointer. Push adds to top, Pop removes from top, Peek reads top without removing.

```
        #include <stdio.h>
        #define MAX 100

        int stack[MAX], top = -1;

        void push(int val) {
            if (top == MAX - 1) { printf("Stack Overflow!\n"); return; }
            stack[++top] = val;
            printf("%d pushed.\n", val);
        }

        int pop() {
            if (top == -1) { printf("Stack Underflow!\n"); return -1; }
            return stack[top--];
        }

        int peek() {
            if (top == -1) { printf("Stack is empty!\n"); return -1; }
            return stack[top];
        }

        void display() {
            if (top == -1) { printf("Stack is empty!\n"); return; }
            printf("Stack (top -> bottom): ");
            for (int i = top; i >= 0; i--) printf("%d ", stack[i]);
            printf("\n");
        }

        int main() {
            int choice, val;
            do {
                printf("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit\nChoice:
");
                scanf("%d", &choice);
                switch (choice) {
                    case 1: printf("Value: "); scanf("%d", &val); push(val);
break;
                    case 2: printf("Popped: %d\n", pop()); break;
                    case 3: printf("Top element: %d\n", peek()); break;
                    case 4: display(); break;
```

```
            }
        } while (choice != 5);
        return 0;
    }
```

Sample Input / Output

```
1.Push  2.Pop  3.Peek  4.Display  5.Exit
Choice: 1  Value: 10  →  10 pushed.
Choice: 1  Value: 20  →  20 pushed.
Choice: 1  Value: 30  →  30 pushed.
Choice: 3  →  Top element: 30
Choice: 4  →  Stack (top -> bottom): 30 20 10
Choice: 2  →  Popped: 30
Choice: 4  →  Stack (top -> bottom): 20 10
Choice: 5  →  Exit
```

SET 04 — Q2: Double Linked List — Insert/Delete at Position, Display

Description

A Doubly Linked List has nodes with `prev`, `data`, and `next` pointers. Insert at position traverses to that node and adjusts both forward and backward links. Delete at position similarly finds and unlinks the node.

```
    #include <stdio.h>
    #include <stdlib.h>

    struct Node {
        int data;
        struct Node *prev, *next;
    };

    struct Node *head = NULL;

    struct Node *newNode(int d) {
        struct Node *n = malloc(sizeof(struct Node));
        n->data = d; n->prev = n->next = NULL;
        return n;
    }

    void insertAtPos(int data, int pos) {
        struct Node *n = newNode(data);
        if (pos == 1) {
            n->next = head;
            if (head) head->prev = n;
            head = n;
            return;
        }
        struct Node *cur = head;
        for (int i = 1; i < pos - 1 && cur; i++) cur = cur->next;
        if (!cur) { printf("Position out of range!\n"); free(n); return;
}
        n->next = cur->next;
        n->prev = cur;
        if (cur->next) cur->next->prev = n;
        cur->next = n;
    }

    void deleteAtPos(int pos) {
        if (!head) { printf("List is empty!\n"); return; }
        struct Node *cur = head;
        if (pos == 1) {
```

```
            head = head->next;
            if (head) head->prev = NULL;
            printf("Deleted %d\n", cur->data);
            free(cur); return;
        }
        for (int i = 1; i < pos && cur; i++) cur = cur->next;
        if (!cur) { printf("Position out of range!\n"); return; }
        if (cur->prev) cur->prev->next = cur->next;
        if (cur->next) cur->next->prev = cur->prev;
        printf("Deleted %d\n", cur->data);
        free(cur);
    }

    void display() {
        struct Node *t = head;
        if (!t) { printf("List is empty!\n"); return; }
        printf("NULL <-> ");
        while (t) { printf("%d <-> ", t->data); t = t->next; }
        printf("NULL\n");
    }

    int main() {
        int ch, d, p;
        do {
            printf("\n1.Insert  2.Delete  3.Display  4.Exit\nChoice: ");
            scanf("%d", &ch);
            switch (ch) {
                case 1:
                    printf("Data: "); scanf("%d", &d);
                    printf("Position: "); scanf("%d", &p);
                    insertAtPos(d, p); break;
                case 2:
                    printf("Position: "); scanf("%d", &p);
                    deleteAtPos(p); break;
                case 3: display(); break;
            }
        } while (ch != 4);
        return 0;
    }
```

Sample Input / Output

```
Choice: 1  Data: 10  Position: 1  →  List: NULL <-> 10 <-> NULL
Choice: 1  Data: 20  Position: 2  →  List: NULL <-> 10 <-> 20 <->
NULL
Choice: 1  Data: 15  Position: 2  →  List: NULL <-> 10 <-> 15 <-> 20
<-> NULL
Choice: 2  Position: 2  →  Deleted 15  →  List: NULL <-> 10 <-> 20
<-> NULL
```

SET 05 — Q1: Queue Using Arrays (Enqueue, Dequeue, Display)

Description

A queue is a FIFO (First In First Out) structure. Elements are added at `rear` and removed from `front`. When rear reaches MAX, the queue is full.

```
    #include <stdio.h>
    #define MAX 100

    int queue[MAX], front = -1, rear = -1;

    void enqueue(int val) {
```

```
            if (rear == MAX - 1) { printf("Queue Full!\n"); return; }
            if (front == -1) front = 0;
            queue[++rear] = val;
            printf("%d enqueued.\n", val);
        }

        int dequeue() {
            if (front == -1 || front > rear) { printf("Queue Empty!\n");
return -1; }
            return queue[front++];
        }

        void display() {
            if (front == -1 || front > rear) { printf("Queue is empty!\n");
return; }
            printf("Queue (front -> rear): ");
            for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
            printf("\n");
        }

        int main() {
            int ch, val;
            do {
                printf("\n1.Enqueue  2.Dequeue  3.Display  4.Exit\nChoice:
");
                scanf("%d", &ch);
                switch (ch) {
                    case 1: printf("Value: "); scanf("%d", &val);
enqueue(val); break;
                    case 2: printf("Dequeued: %d\n", dequeue()); break;
                    case 3: display(); break;
                }
            } while (ch != 4);
            return 0;
        }
```

Sample Input / Output

```
Choice: 1  Value: 10  →  10 enqueued.
Choice: 1  Value: 20  →  20 enqueued.
Choice: 1  Value: 30  →  30 enqueued.
Choice: 3  →  Queue (front -> rear): 10 20 30
Choice: 2  →  Dequeued: 10
Choice: 3  →  Queue (front -> rear): 20 30
```

SET 05 — Q2: Single Linked List — Insert/Delete at Beginning & End, Display

Description

A Singly Linked List has nodes with `data` and `next`. Inserting at beginning updates the head; inserting at end traverses to the last node. Deleting at beginning moves head forward; deleting at end traverses to second-to-last and sets its `next` to NULL.

```
        #include <stdio.h>
        #include <stdlib.h>

        struct Node { int data; struct Node *next; };
        struct Node *head = NULL;

        struct Node *newNode(int d) {
            struct Node *n = malloc(sizeof(struct Node));
            n->data = d; n->next = NULL; return n;
        }
```

```c
        void insertBeg(int d) {
            struct Node *n = newNode(d);
            n->next = head; head = n;
        }

        void insertEnd(int d) {
            struct Node *n = newNode(d);
            if (!head) { head = n; return; }
            struct Node *t = head;
            while (t->next) t = t->next;
            t->next = n;
        }

        void deleteBeg() {
            if (!head) { printf("List Empty!\n"); return; }
            struct Node *t = head;
            head = head->next;
            printf("Deleted %d\n", t->data); free(t);
        }

        void deleteEnd() {
            if (!head) { printf("List Empty!\n"); return; }
            if (!head->next) {
                printf("Deleted %d\n", head->data); free(head); head = NULL;
return;
            }
            struct Node *t = head;
            while (t->next->next) t = t->next;
            printf("Deleted %d\n", t->next->data);
            free(t->next); t->next = NULL;
        }

        void display() {
            struct Node *t = head;
            if (!t) { printf("List Empty!\n"); return; }
            while (t) { printf("%d -> ", t->data); t = t->next; }
            printf("NULL\n");
        }

        int main() {
            int ch, d;
            do {
                printf("\n1.InsBeg 2.InsEnd 3.DelBeg 4.DelEnd 5.Display
6.Exit\nChoice: ");
                scanf("%d", &ch);
                switch (ch) {
                    case 1: printf("Data: "); scanf("%d", &d); insertBeg(d);
break;
                    case 2: printf("Data: "); scanf("%d", &d); insertEnd(d);
break;
                    case 3: deleteBeg(); break;
                    case 4: deleteEnd(); break;
                    case 5: display(); break;
                }
            } while (ch != 6);
            return 0;
        }
```

Sample Input / Output

```
Choice: 1  Data: 10  →  10 -> NULL
Choice: 2  Data: 20  →  10 -> 20 -> NULL
Choice: 1  Data: 5   →  5 -> 10 -> 20 -> NULL
Choice: 3  →  Deleted 5  →  10 -> 20 -> NULL
Choice: 4  →  Deleted 20  →  10 -> NULL
```

SET 06 — Q1: Circular Queue Using Arrays

Description

In a Circular Queue, the last position wraps back to the first position forming a circle. This avoids the “false full” problem of a linear queue. Uses modulo arithmetic.

```c
#include <stdio.h>
#define MAX 5

int cq[MAX], front = -1, rear = -1;

int isFull()  { return (front == 0 && rear == MAX-1) || (rear == (front-1) % (MAX-1)); }
int isEmpty() { return front == -1; }

void enqueue(int val) {
    if (isFull()) { printf("Circular Queue Full!\n"); return; }
    if (front == -1) front = rear = 0;
    else rear = (rear + 1) % MAX;
    cq[rear] = val;
    printf("%d enqueued.\n", val);
}

int dequeue() {
    if (isEmpty()) { printf("Circular Queue Empty!\n"); return -1; }
    int val = cq[front];
    if (front == rear) front = rear = -1;
    else front = (front + 1) % MAX;
    return val;
}

void display() {
    if (isEmpty()) { printf("Queue is empty!\n"); return; }
    printf("Circular Queue: ");
    int i = front;
    while (1) {
        printf("%d ", cq[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int ch, val;
    do {
        printf("\n1.Enqueue  2.Dequeue  3.Display  4.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &val); enqueue(val); break;
            case 2: printf("Dequeued: %d\n", dequeue()); break;
            case 3: display(); break;
        }
    } while (ch != 4);
    return 0;
}
```

Sample Input / Output

```
Choice: 1  Value: 10  →  10 enqueued.
Choice: 1  Value: 20  →  20 enqueued.
Choice: 1  Value: 30  →  30 enqueued.
```

```
Choice: 3  →  Circular Queue: 10 20 30
Choice: 2  →  Dequeued: 10
Choice: 1  Value: 40  →  40 enqueued.
Choice: 3  →  Circular Queue: 20 30 40
```

SET 07 — Q1: Single Linked List — Insert at Beginning, End, Middle; Delete; Display

Description

Extended Single Linked List supporting insertion at any specific position (middle). Traverses to position-1 and re-links. Deletion by value searches and unlinks.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };
struct Node *head = NULL;

struct Node *newNode(int d) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = d; n->next = NULL; return n;
}

void insertBeg(int d)  { struct Node *n = newNode(d); n->next =
head; head = n; }

void insertEnd(int d) {
    struct Node *n = newNode(d);
    if (!head) { head = n; return; }
    struct Node *t = head;
    while (t->next) t = t->next;
    t->next = n;
}

void insertMid(int d, int pos) {
    if (pos == 1) { insertBeg(d); return; }
    struct Node *n = newNode(d), *t = head;
    for (int i = 1; i < pos - 1 && t; i++) t = t->next;
    if (!t) { printf("Position out of range!\n"); free(n); return; }
    n->next = t->next; t->next = n;
}

void deleteVal(int d) {
    if (!head) { printf("List empty!\n"); return; }
    if (head->data == d) {
        struct Node *t = head; head = head->next;
        printf("Deleted %d\n", t->data); free(t); return;
    }
    struct Node *t = head;
    while (t->next && t->next->data != d) t = t->next;
    if (!t->next) { printf("Value %d not found!\n", d); return; }
    struct Node *del = t->next;
    t->next = del->next;
    printf("Deleted %d\n", del->data); free(del);
}

void display() {
    struct Node *t = head;
    if (!t) { printf("List empty!\n"); return; }
    while (t) { printf("%d -> ", t->data); t = t->next; }
    printf("NULL\n");
}
```

```c
        int main() {
            int ch, d, p;
            do {
                printf("\n1.InsBeg 2.InsEnd 3.InsMid 4.Delete 5.Display
6.Exit\nChoice: ");
                scanf("%d", &ch);
                switch (ch) {
                    case 1: printf("Data: "); scanf("%d",&d); insertBeg(d);
break;
                    case 2: printf("Data: "); scanf("%d",&d); insertEnd(d);
break;
                    case 3: printf("Data: "); scanf("%d",&d); printf("Pos:
"); scanf("%d",&p); insertMid(d,p); break;
                    case 4: printf("Value to delete: "); scanf("%d",&d);
deleteVal(d); break;
                    case 5: display(); break;
                }
            } while (ch != 6);
            return 0;
        }
```

Sample Input / Output

```
Choice: 2  Data: 10  →  10 -> NULL
Choice: 2  Data: 30  →  10 -> 30 -> NULL
Choice: 3  Data: 20  Pos: 2  →  10 -> 20 -> 30 -> NULL
Choice: 1  Data: 5   →  5 -> 10 -> 20 -> 30 -> NULL
Choice: 4  Value: 20  →  Deleted 20  →  5 -> 10 -> 30 -> NULL
```

SET 07 — Q2: Queue Using Linked List

Description

Dynamic queue using Linked List. No size limit. `front` pointer for dequeue (head), `rear` pointer for enqueue (tail). More memory-efficient than fixed array queue.

```c
        #include <stdio.h>
        #include <stdlib.h>

        struct Node { int data; struct Node *next; };
        struct Node *front = NULL, *rear = NULL;

        void enqueue(int val) {
            struct Node *n = malloc(sizeof(struct Node));
            n->data = val; n->next = NULL;
            if (!rear) { front = rear = n; }
            else { rear->next = n; rear = n; }
            printf("%d enqueued.\n", val);
        }

        int dequeue() {
            if (!front) { printf("Queue empty!\n"); return -1; }
            struct Node *t = front;
            int val = t->data;
            front = front->next;
            if (!front) rear = NULL;
            free(t);
            return val;
        }

        void display() {
            if (!front) { printf("Queue empty!\n"); return; }
            struct Node *t = front;
            printf("Queue (front -> rear): ");
```

```
            while (t) { printf("%d ", t->data); t = t->next; }
            printf("\n");
        }

        int main() {
            int ch, val;
            do {
                printf("\n1.Enqueue  2.Dequeue  3.Display  4.Exit\nChoice:
");
                scanf("%d", &ch);
                switch (ch) {
                    case 1: printf("Value: "); scanf("%d",&val);
enqueue(val); break;
                    case 2: printf("Dequeued: %d\n", dequeue()); break;
                    case 3: display(); break;
                }
            } while (ch != 4);
            return 0;
        }
```

Sample Input / Output

```
Choice: 1  Value: 100  →  100 enqueued.
Choice: 1  Value: 200  →  200 enqueued.
Choice: 3  →  Queue (front -> rear): 100 200
Choice: 2  →  Dequeued: 100
Choice: 3  →  Queue (front -> rear): 200
```

SET 08 — Q1 / SET 10 — Q2: BST Traversals (Inorder, Preorder, Postorder)

Description

Binary Search Tree traversals: - **Inorder** (Left → Root → Right): gives sorted output - **Preorder** (Root → Left → Right): used for tree copying - **Postorder** (Left → Right → Root): used for tree deletion

```
        #include <stdio.h>
        #include <stdlib.h>

        struct Node {
            int data;
            struct Node *left, *right;
        };

        struct Node *newNode(int d) {
            struct Node *n = malloc(sizeof(struct Node));
            n->data = d; n->left = n->right = NULL; return n;
        }

        struct Node *insert(struct Node *root, int d) {
            if (!root) return newNode(d);
            if (d < root->data) root->left  = insert(root->left,  d);
            else if (d > root->data) root->right = insert(root->right, d);
            return root;
        }

        void inorder(struct Node *r)   { if (r) { inorder(r->left);
printf("%d ", r->data); inorder(r->right);  } }
        void preorder(struct Node *r)  { if (r) { printf("%d ", r->data);
preorder(r->left);  preorder(r->right); } }
        void postorder(struct Node *r) { if (r) { postorder(r->left);
postorder(r->right); printf("%d ", r->data); } }

        int main() {
```

```c
    struct Node *root = NULL;
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
    printf("Inorder   : "); inorder(root);   printf("\n");
    printf("Preorder  : "); preorder(root);  printf("\n");
    printf("Postorder : "); postorder(root); printf("\n");
    return 0;
}
```

Sample Input

```
Enter number of nodes: 7
Enter values: 50 30 70 20 40 60 80
```

Sample Output

```
Inorder   : 20 30 40 50 60 70 80
Preorder  : 50 30 20 40 70 60 80
Postorder : 20 40 30 60 80 70 50
```

SET 08 — Q2 / SET 10 — Q1: Copy One Structure Variable to Another

Description

In C, a structure can be copied directly using the assignment operator =. This performs a **shallow copy** — all field values are duplicated.

```c
#include <stdio.h>
#include <string.h>

struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s1, s2;

    printf("Enter Roll No : "); scanf("%d", &s1.roll);
    printf("Enter Name    : "); scanf("%s", s1.name);
    printf("Enter Marks   : "); scanf("%f", &s1.marks);

    // Direct structure copy using assignment
    s2 = s1;

    printf("\n--- Original (s1) ---\n");
    printf("Roll : %d\nName : %s\nMarks: %.2f\n", s1.roll, s1.name,
s1.marks);

    printf("\n--- Copied (s2) ---\n");
    printf("Roll : %d\nName : %s\nMarks: %.2f\n", s2.roll, s2.name,
s2.marks);

    // Modify s2 to show they are independent
    s2.roll = 99;
    strcpy(s2.name, "Copy");
```

```
            printf("\nAfter modifying s2:\n");
            printf("s1.roll = %d | s2.roll = %d\n", s1.roll, s2.roll);

            return 0;
        }
```

Sample Input

```
Enter Roll No : 5
Enter Name    : Alice
Enter Marks   : 92.5
```

Sample Output

```
--- Original (s1) ---
Roll : 5
Name : Alice
Marks: 92.50

--- Copied (s2) ---
Roll : 5
Name : Alice
Marks: 92.50

After modifying s2:
s1.roll = 5 | s2.roll = 99
```

SET 09 — Q2: BST — Insert, Search, and Delete

Description

Full BST implementation: - **Insert**: Compare with root, go left if smaller, right if larger. - **Search**: Traverse comparing target with current node. - **Delete**: 3 cases — leaf node, one child, two children (replace with inorder successor).

```
        #include <stdio.h>
        #include <stdlib.h>

        struct Node {
            int data;
            struct Node *left, *right;
        };

        struct Node *newNode(int d) {
            struct Node *n = malloc(sizeof(struct Node));
            n->data = d; n->left = n->right = NULL; return n;
        }

        struct Node *insert(struct Node *root, int d) {
            if (!root) return newNode(d);
            if (d < root->data) root->left  = insert(root->left,  d);
            else if (d > root->data) root->right = insert(root->right, d);
            else printf("Duplicate %d not inserted.\n", d);
            return root;
        }

        struct Node *search(struct Node *root, int d) {
            if (!root || root->data == d) return root;
            return (d < root->data) ? search(root->left, d) : search(root-
>right, d);
        }
```

```
        struct Node *minNode(struct Node *r) {
            while (r->left) r = r->left;
            return r;
        }

        struct Node *deleteNode(struct Node *root, int d) {
            if (!root) { printf("Value %d not found!\n", d); return root; }
            if (d < root->data) root->left  = deleteNode(root->left,  d);
            else if (d > root->data) root->right = deleteNode(root->right,
d);
            else {
                // Node found
                if (!root->left)  { struct Node *t = root->right;
free(root); return t; }
                if (!root->right) { struct Node *t = root->left;
free(root); return t; }
                // Two children: replace with inorder successor
                struct Node *succ = minNode(root->right);
                root->data  = succ->data;
                root->right = deleteNode(root->right, succ->data);
            }
            return root;
        }

        void inorder(struct Node *r) {
            if (r) { inorder(r->left); printf("%d ", r->data); inorder(r-
>right); }
        }

        int main() {
            struct Node *root = NULL;
            int ch, val;
            do {
                printf("\n1.Insert  2.Search  3.Delete  4.Display
5.Exit\nChoice: ");
                scanf("%d", &ch);
                switch (ch) {
                    case 1:
                        printf("Value: "); scanf("%d",&val);
                        root = insert(root, val);
                        break;
                    case 2:
                        printf("Search: "); scanf("%d",&val);
                        printf(search(root, val) ? "%d Found!\n" : "%d Not
Found!\n", val);
                        break;
                    case 3:
                        printf("Delete: "); scanf("%d",&val);
                        root = deleteNode(root, val);
                        printf("Deleted %d\n", val);
                        break;
                    case 4:
                        printf("Inorder: "); inorder(root); printf("\n");
                        break;
                }
            } while (ch != 5);
            return 0;
        }
```

Sample Input / Output

```
Choice: 1  Value: 50  →  (insert)
Choice: 1  Value: 30  →  (insert)
Choice: 1  Value: 70  →  (insert)
Choice: 1  Value: 20  →  (insert)
Choice: 4  →  Inorder: 20 30 50 70
Choice: 2  Search: 30  →  30 Found!
Choice: 2  Search: 99  →  99 Not Found!
```

```
Choice: 3  Delete: 30  →  Deleted 30
Choice: 4  →  Inorder: 20 50 70
Choice: 5  →  Exit
```

Quick Reference — All Sets Mapped

Set	Q1	Q2
01	Total & Average (Structures)	Merge Sort
02	Postfix Evaluation	Quick Sort
03	Command Line Student Details	Radix Sort
04	Stack (Array)	Doubly Linked List
05	Queue (Array)	Singly Linked List
06	Circular Queue	Stack (Array) ← same as Set 04 Q1
07	Single LL (all ops)	Queue (Linked List)
08	BST Traversals	Copy Structure
09	Total & Average ← same as Set 01 Q1	BST Insert/Search/Delete
10	Copy Structure ← same as Set 08 Q2	BST Traversals ← same as Set 08 Q1

> Note: Repeated questions across sets share the same answer.